

1 Project Overview

In this project, each team will design and implement a working software or AI-powered service by actively using generative AI tools. The goal is to experience a modern AI-assisted development workflow, often called “vibe coding,” while still demonstrating a clear understanding of the system you submit.

You are encouraged to use tools such as ChatGPT, Claude, Cursor, GitHub Copilot, Gemini, or other AI-assisted programming tools. However, this project is not a simple “ask AI to build everything and submit the result” assignment. You are expected to use AI as a development partner while taking responsibility for problem definition, service design, system design, code validation, debugging, integration, and explanation.

The core of this project is service design. Your team should not simply connect an AI API to a user interface. You should design a service that addresses a real user need, provides a clear interaction flow, and delivers visible value through its features. The final service does not need to be commercially complete, but it must be functional enough to demonstrate its main value.

The key principle of this project is:

AI may help you generate code, design interfaces, debug errors, and prepare documentation. However, you are responsible for the final system. If you cannot explain your own project, it will not be considered your own work.

2 Project Objectives

By completing this project, you should be able to:

- define a practical problem and translate it into a service concept,
- analyze target users and design a realistic user scenario,
- design a coherent service flow from user input to system output,
- use generative AI tools effectively for software development,
- build a working prototype with frontend, backend, AI API, database, or automation components,
- inspect, modify, and validate AI-generated code,
- explain the architecture, data flow, core implementation, and design decisions of the system,
- and demonstrate the final service clearly through a live demo.

3 AI Tool Usage Policy

AI tools are **allowed and encouraged** in this project. You may use AI tools for:

- brainstorming project ideas,
- designing service scenarios and user flows,
- generating initial UI or backend code,
- implementing API calls,
- debugging error messages,
- writing test cases,

- improving code readability,
- designing database schemas,
- preparing README drafts,
- and preparing presentation outlines.

However, the following behaviors are **not acceptable**:

- submitting AI-generated code without understanding it,
- reading an AI-generated presentation script word-for-word,
- failing to explain the main functions, files, or data flow of the project,
- claiming AI-generated or copied work as entirely self-written without disclosure,
- submitting a project that does not run and blaming the AI tool,
- and using external templates, repositories, datasets, or APIs without proper attribution.

You are not graded on whether every line of code was manually written by you. You are graded on whether you can design, build, adapt, verify, and explain a working service using modern AI-assisted development practices.

4 Project Scope

Your project must result in a working software or AI-powered service. Possible project types include:

- AI chatbot service,
- personal productivity assistant,
- class material summarizer,
- quiz generation service,
- travel planning assistant,
- food or exercise recommendation service,
- campus information chatbot,
- research paper summarization tool,
- image classification demo service,
- voice memo summarization service,
- coding error explanation assistant,
- news summarization and keyword analysis dashboard,
- small business or community support web application,
- or another service idea approved by the instructor.

Regardless of the topic, your project should be designed as a service, not just as a collection of functions. You should clearly define who will use the service, what problem they have, what input they provide, what output they receive, and why the result is useful.

The project should include at least one meaningful software component, such as:

- a web interface,
- a backend API,
- an AI model or LLM API integration,
- a database,

- a data processing pipeline,
- an automation workflow,
- or a dashboard or visualization.

Your service should also have a clear user flow from user input, to system or AI processing, to meaningful output or recommendation.

5 Recommended Technology Stacks

You may choose your own stack. The following combinations are recommended.

5.1 Beginner-Friendly Stack

- Python + Streamlit
- Python + Gradio
- Python + Flask
- OpenAI API, Gemini API, Hugging Face API, or local model API
- SQLite or simple file-based storage

5.2 Intermediate Stack

- React + FastAPI
- React + Node.js/Express
- Next.js + API routes
- Supabase, Firebase, or PostgreSQL
- LLM API or custom ML model integration

5.3 Deployment Options

- Streamlit Cloud
- Hugging Face Spaces
- Vercel
- Render
- Railway
- Local execution with a clear README

Deployment is recommended but not mandatory. If your service is not deployed online, it must run reliably on a local machine using the instructions in your README file.

6 Required Deliverables

Each team must submit the following items.

6.1 Working Service

Submit either:

- a deployed service URL, or
- a GitHub repository with clear local execution instructions.

The service must be demonstrable during the final presentation.

6.2 GitHub Repository

Your repository should include:

- source code,
- README.md,
- requirements.txt, package.json, environment.yml, or an equivalent dependency file,
- sample input or test data if needed,
- screenshots or demo images if helpful,
- and AI_USAGE.md.

6.3 Technical Report

Submit a short PDF report. A recommended structure is:

1. Introduction
2. Problem Definition
3. Target Users and User Scenario
4. Service Design Rationale
5. Service Features and User Flow
6. System Architecture
7. Implementation Details
8. AI-Assisted Development Process
9. Testing and Demonstration
10. Discussion and Future Improvements
11. References

The report should explain not only what your system implements, but also why the service is designed in that way. For example, explain why the selected features are useful for the target users and how the service output helps them.

6.4 AI Usage Report

Each team must submit an AI_USAGE.md file. This file is used to evaluate how thoughtfully and responsibly you used AI tools. Your AI_USAGE.md should include:

- AI tools used,
- tasks supported by AI,
- representative prompts,
- AI-generated outputs that you modified,
- core files that your team can explain,
- and lessons learned from AI-assisted development.

7 AI Usage Report Example

The following is an example structure for `AI_USAGE.md`.

```
# AI Usage Report

## AI Tools Used
- ChatGPT
- Claude
- Cursor
- GitHub Copilot

## Tasks Supported by AI
- Brainstormed project ideas
- Designed user scenarios and service flow
- Generated initial React components
- Created a FastAPI backend template
- Helped debug API connection errors
- Suggested README structure

## Example Prompts
1. "Create a FastAPI backend for a task management app."
2. "Explain why this React state update does not work."
3. "Refactor this function to make it easier to understand."

## AI Outputs We Modified
- Added input validation to the generated backend code
- Changed the database schema
- Reorganized the UI components
- Fixed incorrect API response handling
- Added error messages for users

## Core Files We Can Explain
- app/main.py
- src/components/Dashboard.jsx
- src/api/client.js
- database/schema.sql

## What We Learned
- We learned how frontend and backend communicate through REST APIs.
- We learned that AI-generated code often requires debugging and validation.
- We learned how to structure a service around user scenarios.
```

8 Final Presentation Guide

The final presentation should focus on the actual service and the design decisions behind it. Do not simply read a script. You should demonstrate, explain, and defend your system, including why your service is useful for the target users. A recommended presentation structure is:

1. **Problem, Target Users, and Service Design**

Explain what problem you are solving, who the service is for, when and how the service will be used, and why your design is appropriate for the target users.

2. **Service Demo**

Show the working service with a realistic user scenario. Make clear what the user inputs, what the system does, what output is produced, and why the output is useful.

3. **System Architecture**

Explain the main components and data flow.

4. **Core Implementation**

Open at least one important code file and explain it.

5. **AI-Assisted Development Process**

Explain how AI helped and what your team changed or improved.

6. **Limitations and Future Work**

Explain what remains incomplete and how the service could be improved.

7. **Q&A**

Answer questions about design, code, implementation choices, and individual contributions.

Each team must explain at least one core code file during the presentation. The explanation should cover what the code does, why it is needed, and how it connects to the rest of the system.

Students may prepare notes or bullet points, but they should not read a full script during the presentation. The presentation should show that each team understands the service, the implementation, and the design decisions behind the project.

9 Individual Understanding Check

For team projects, every member must be able to explain their own contribution. During Q&A, the instructor may ask individual students to explain:

- which part they implemented,
- which files they worked on,
- what errors they encountered,
- how they used AI tools,
- how they verified that the generated code was correct,
- how data flows through the service,
- why the service design supports the target user scenario,
- and how the main function, component, or API endpoint works.

A team may have a working demo, but individual grades may differ if contributions and understanding differ significantly.

10 Project Milestones

10.1 Milestone 1: Service Proposal

Submit a short proposal including:

- project title,
- problem statement,
- target users,
- user scenario,
- three core features,
- expected AI component,
- planned technology stack,
- and team member roles.

10.2 Milestone 2: Prototype

Submit a prototype with:

- basic working interface,
- at least one implemented core feature,
- initial GitHub repository,
- short progress note,
- and preliminary AI_USAGE.md.

10.3 Milestone 3: Final Submission

Submit:

- final service or runnable repository,
- final README.md,
- final AI_USAGE.md,
- technical report PDF,
- presentation slides,
- and demo-ready system.

11 Grading Criteria

The project will be evaluated based on the following criteria:

- **Problem definition and service design (15%)**
clarity of the problem, specificity of target users, realism of the user scenario, quality of the service flow, and meaningfulness of the service idea
- **Service completeness (25%)**
correctness of execution, implementation of main features, quality of demo, usability of the service, and how well the service supports the intended user scenario
- **Technical implementation and architecture (20%)**
quality of system structure, integration of frontend/backend/API/database/model components, and maintainability of code
- **Effective use of AI tools (15%)**
thoughtful use of AI for design, implementation, debugging, or documentation; clear explanation of what was generated, modified, and verified
- **Understanding and explanation (15%)**
ability to explain system architecture, data flow, core code, service design, and implementation decisions without relying on a prepared AI-generated script
- **Presentation and demonstration quality (10%)**
clarity, organization, demo quality, effective communication of the value of the service, and ability to explain the service without simply reading a script

12 Submission Checklist

Before submission, check the following items:

- The service runs on a clean environment or is deployed online.
- The README explains how to install and run the project.
- The target users and user scenario are clearly explained.
- The service flow from user input to final output is easy to understand.
- The main features are connected to the needs of the target users.
- The main features are demonstrated with realistic examples.
- The repository does not expose private API keys.
- Environment variables are documented properly.
- AI_USAGE.md clearly explains how AI tools were used.
- Each team member can explain their contribution.
- At least one core code file can be explained during the presentation.
- External sources, templates, datasets, and APIs are properly credited.
- The presentation is demo-oriented, not script-reading-oriented.

13 Important Notes

- AI-assisted development is allowed and encouraged, but blind copying is not acceptable.
- A good project is not just a working program. It should be a well-designed service that connects a real problem, target users, useful features, and a clear user experience.
- A working service with clear explanation is better than an ambitious but poorly understood project.
- Do not build features only because AI can generate them. Choose features because they help the target user solve the problem more effectively.
- Your demo should show realistic use, not only a trivial example.
- Do not expose private API keys or credentials in your GitHub repository.
- If you use external code, templates, datasets, or APIs, cite them clearly.
- Every team member should be ready to answer technical questions about the project.
- Good projects show not only functionality, but also careful reasoning, validation, and communication.